ClaimGuard AI

System Architecture Plan

AI-Assisted Health Insurance Claim Rejection Verification and Evidence-Based Appeal Support for India

Document status	Updated architecture baseline
Version	2026.10 - post safe-analysis audit
Updated through	01 October 2026
Scope	Current backend, frontend, extraction, evidence, rule, financial and validation architecture

Engineering principle

AI reads the case; deterministic rules verify the case. AI/VLM/OCR is responsible for extracting and interpreting document content. Deterministic policy rules, dependency logic and financial calculations are responsible for auditable verification. Uncertain or incomplete evidence routes to review instead of being guessed.

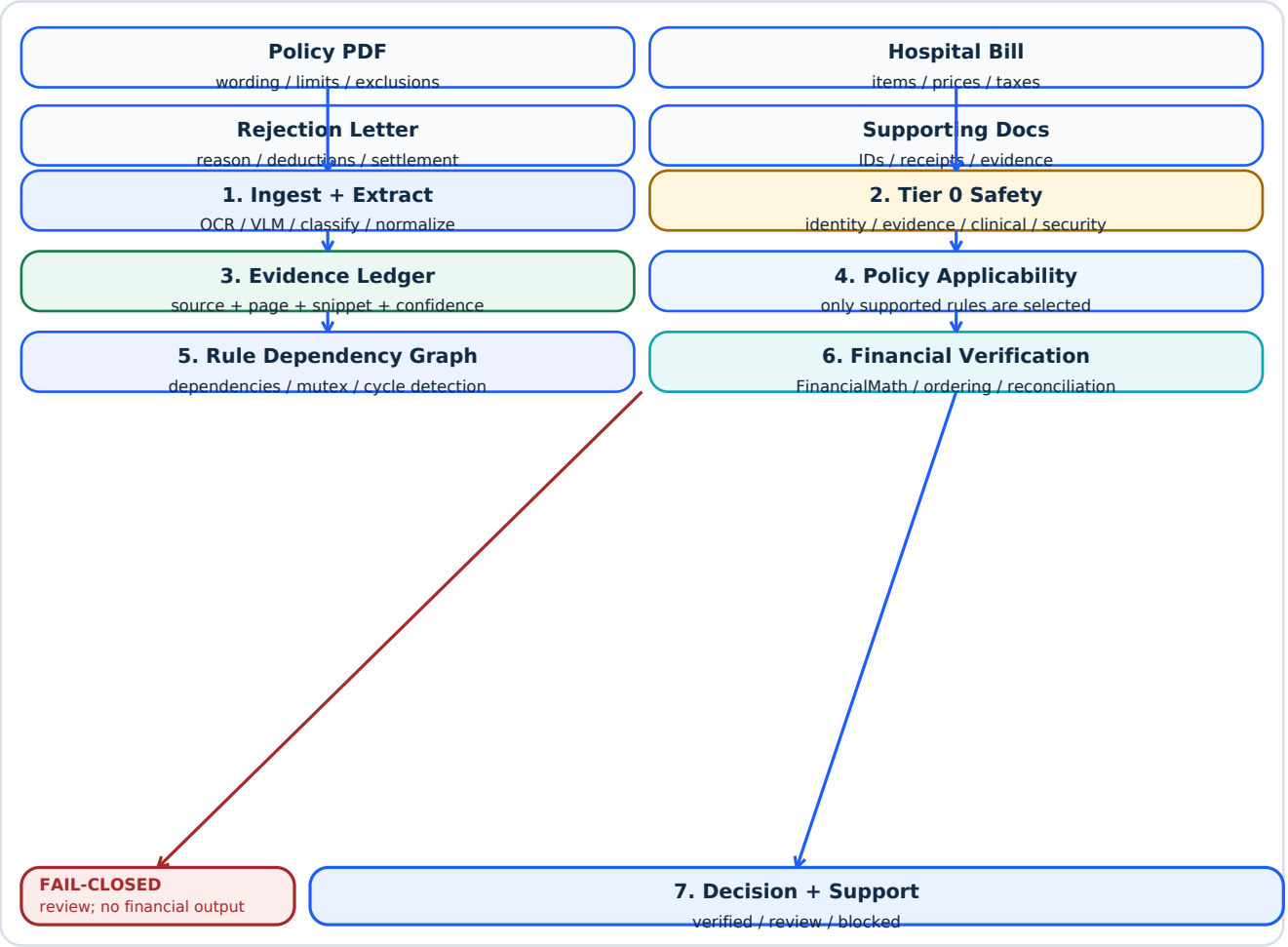
Current validation status

Automated backend tests	315 passed, 2 skipped
Frontend tests	77 passed, 0 failed
FastAPI startup import	Successful
Production frontend build	Successful; Vite reports a large JavaScript chunk warning
Live VLM/provider validation	Not yet verified with real documents and configured credentials
Broader provenance	Key financial inputs covered; policy-clause/nonfinancial provenance needs further validation

Prepared as an engineering architecture and validation plan based on the current ClaimGuard design, audit findings and implementation status recorded in the project conversation.

1. Architecture Overview

ClaimGuard is a post-rejection verification system. It does not attempt to replace an insurer, physician, lawyer or claims professional. Its purpose is to transform heterogeneous claim documents into traceable evidence, apply only relevant policy and regulatory logic, independently verify calculations, expose discrepancies, and produce structured appeal support when the evidence justifies it.



System boundary

- In scope: document ingestion, classification, OCR/VLM extraction, structured normalization, evidence/provenance, Tier 0 safety, policy applicability, deterministic rules, financial reconciliation, findings, audit trail, review states, and appeal support.
- Out of scope: autonomous clinical-necessity decisions, unsupported legal conclusions, invented claim data, or autonomous live financial transactions.
- Safety posture: missing, conflicting or low-confidence critical evidence causes a fail-closed result instead of allowing downstream financial adjudication.

2. Document Map

This document describes the current architecture from the document boundary to the user-facing decision state, followed by validation, known limitations and the next hardening steps.

3. Problem and Design Goals	Define the exact decision-support problem and constraints.
4. End-to-End Data Flow	Show how documents become verified evidence and findings.
5. Extraction and Normalization	Describe OCR/VLM processing and fail-closed extraction contracts.
6. Tier 0 Safety Gates	Explain when financial processing is allowed to proceed.
7. Evidence and Provenance	Define the evidence ledger, source binding and confidence rules.
8. Policy Applicability and Rule Engine	Describe applicable-rule selection and deterministic verification.
9. Rule Dependency Graph and FinancialMath	Explain rule ordering, conflict control and financial reconciliation.
10. Adjudication State and Outcomes	Separate processing lifecycle from authoritative analysis outcome.
11. Frontend and API Safety	Prevent missing API fields or UI fallbacks from fabricating results.
12. Security and Document Integrity	Cover prompt injection, credentials, hashing and forensic states.
13. Test and UAT Strategy	Summarize current test coverage and future validation.
14. Current Implementation Status	Record completed repairs and remaining limits.
15. Research Grounding	Map architecture concepts to the selected research references.
16. Next Engineering Phase	Prioritize live-provider validation and broader provenance.

Status legend

Verified in current automated validation	Implemented and covered by the recorded test/build/import checks.
Implemented, live validation pending	Code path exists but real provider/documents or broader evidence validation is still outstanding.
Future hardening	Not yet part of the verified current baseline.

3. Problem and Design Goals

A health-insurance claim can be rejected because a condition was considered inapplicable, a limit was interpreted incorrectly, a deduction was calculated incorrectly, or required evidence was incomplete. The operational difficulty is not simply reading one document. The system must connect the policy, the claim context, the hospital bill, the rejection/settlement letter and supporting evidence without silently filling gaps.

Primary design goals

- Cross-document verification: compare identity, dates, claim identifiers, amounts and reasons across documents.
- Policy-grounded reasoning: determine which clauses are applicable before executing rule logic.
- Deterministic financial verification: reproduce arithmetic with explicit ordering and reconciliation instead of trusting LLM arithmetic.
- Evidence traceability: bind accepted values to source documents, pages, snippets, confidence and transformations.
- Fail-closed behavior: a missing critical input stops adjudication rather than being converted to zero, a default value or a demo value.
- Human review for uncertainty: ambiguous, conflicting or low-confidence cases become review states, not guessed decisions.

Non-goals

- Autonomous clinical necessity determination. The clinical firewall prevents ClaimGuard from presenting clinical judgment as a deterministic policy calculation.
- Autonomous legal representation. ClaimGuard supplies evidence-based appeal support, not a binding legal opinion.
- Autonomous settlement or payment. The current system is a verification and decision-support workflow.

4. End-to-End Data Flow

The pipeline is intentionally staged so that each layer has a narrow responsibility. A later layer must not create missing facts that an earlier layer failed to establish.

1. Upload / ingest	Files	Store files, identify document types, calculate hashes and preserve document IDs.	Unreadable file; unsupported type; missing upload.
2. Extraction	PDF/image	OCR/VLM extraction with all relevant PDF pages handed to multimodal processing.	Missing credentials; provider error; empty critical extraction.
3. Normalization	Raw extraction	Map fields into structured models without inventing values.	Schema mismatch; critical field missing.
4. Tier 0	Structured evidence	Check identity, evidence, clinical boundary, conflicts, security and processing safety.	Identity conflict; insufficient evidence; prompt injection.
5. Evidence ledger	Accepted fields	Persist value, normalized value, source document, page, section, snippet, bbox, confidence and transformations.	No page/snippet/confidence for critical financial value.
6. Applicability	Policy + claim context	Select only conditions supported by policy wording and claim facts.	Contradictory policy context; missing rule inputs.
7. Rule execution	Applicable rules	Evaluate dependencies, mutual exclusions and deterministic conditions.	Cycle; mutex conflict; unresolved dependency.
8. Financial verification	Verified amounts	Use FinancialMath, explicit ordering and independent reconciliation.	Arithmetic mismatch; unverified amount; blocked rule.
9. Outcome	Verified findings	Return verified findings, review/blocked state, or extraction failure.	Never fabricate payout, delta or appeal text.
10. UI / appeal support	API result	Render only backend-authoritative data and evidence-backed explanation.	Never replace null with demo claim or estimated billed amount.

Critical invariant

A later layer may transform verified evidence, but it may not create a new financial fact merely because a downstream schema or UI expects one.

5. Extraction and Normalization

Extraction is the probabilistic boundary of ClaimGuard. The current repair specifically ensures that provider failure, missing credentials or incomplete extraction is observable and cannot masquerade as a complete claim.

Current extraction contract

Successful extraction: structured data plus diagnostic/provenance information sufficient for downstream verification.

Failed extraction: `source="none"`, `data=None`, an explicit failure/review status, and diagnostic issues. OCR text may remain available as unstructured diagnostic content, but it does not become a substitute for successful structured evidence.

PDF handling

- The first page may be retained for document classification, but multimodal extraction is handed all rendered PDF pages so that later pages containing exclusions, tables or settlement details are not silently ignored.
- Critical extracted values must survive normalization with the source document identity intact.
- Missing fields remain missing. The system does not convert absent amounts into 0 or fabricate defaults.

Critical financial field acceptance

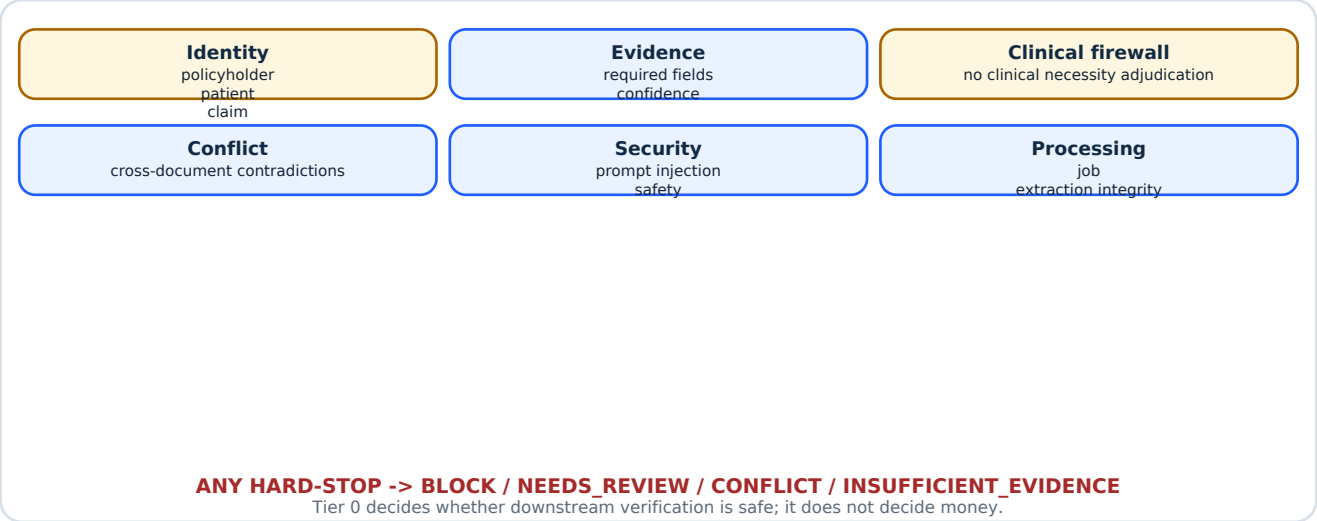
For key financial inputs, acceptance into the evidence ledger requires a page citation, a source snippet that matches the OCR representation, and field confidence. The evidence entry is bound to the uploaded document ID/hash so the value is traceable to the exact source artifact.

Design consequence

The extraction boundary is intentionally conservative. A system that says "I do not have enough evidence" is preferred to a system that produces a plausible but unsupported payout number.

6. Tier 0 Safety Gates

Tier 0 is the highest-priority pre-adjudication layer. It does not decide money. It decides whether the claim is sufficiently safe and evidenced for downstream deterministic verification.



Gate semantics

Identity	Are policyholder, patient and claim context sufficiently supported and role-consistent?	CONFLICT or INSUFFICIENT_EVIDENCE
Evidence	Are required fields present with enough confidence and provenance?	NEEDS_REVIEW / BLOCKED
Clinical firewall	Does the rejection require a medical-necessity judgment beyond the system boundary?	Escalate; do not adjudicate clinically
Conflict	Do documents disagree on key identifiers, dates or amounts?	CONFLICT
Security	Is extracted text attempting to alter system instructions or execute an unsafe instruction?	Isolate content; stop unsafe path
Processing	Did the extraction/job lifecycle complete legitimately?	EXTRACTION_FAILED or review

The Tier 0 gate is therefore a trust boundary between probabilistic document understanding and deterministic financial verification.

7. Evidence and Provenance

ClaimGuard treats each important extracted fact as an evidence object rather than a free-floating database value. This is essential for explaining not only what was concluded, but why the system considered a specific number or clause trustworthy enough to use.

Provenance record - current target shape

```
value, normalized_value, source_document_id, source_document_type, source_hash, page, section, bounding_box,
source_text, extraction_confidence, extraction_model, model_version, timestamp, transformations
```

Financial evidence acceptance

- Page citation is mandatory for critical financial values.
- The captured source snippet must match the OCR/evidence representation sufficiently to support auditability.
- Field confidence is recorded and used as a gate rather than being decorative metadata.
- Accepted evidence is tied to uploaded document IDs/hashes so later analysis cannot silently substitute a different source artifact.

Audit-chain semantics

An empty audit chain is not interpreted as a valid chain of evidence. When forensic analysis did not run, the UI displays NOT_ANALYZED rather than a zero score that could be mistaken for a completed check.

Current limitation

Provenance coverage is currently strongest for key financial inputs. Policy-clause provenance and nonfinancial field provenance still require broader validation before the architecture should be considered fully end-to-end evidenced.

8. Policy Applicability and Deterministic Rule Engine

The rules layer must distinguish between "a rule exists" and "the rule applies to this claim." Applicability is resolved from the policy wording and claim context before the rule is executed.

Core rule families

- Eligibility and timing: waiting-period checks, policy start dates, exact calendar arithmetic, 60-month moratorium logic and portability credits where supported by evidence.
- Coverage and limits: room-rent limits, sub-limits, exclusions and policy-specific caps.
- Cost sharing: deductible and co-pay application according to policy-specified ordering and scope.
- Financial reconciliation: item totals, subtotals, taxes, discounts, charges, deductions and final payable amounts.
- Identity/context: policyholder vs patient role handling, claim number and document identity consistency.

Examples of deterministic calculation buckets

Quantity x unit price = item total; sum of items = subtotal; CGST + SGST = tax; subtotal - discount + taxes + charges = grand total. These relationships are calculated by deterministic financial utilities, not by an LLM.

Critical policy interpretation requirement

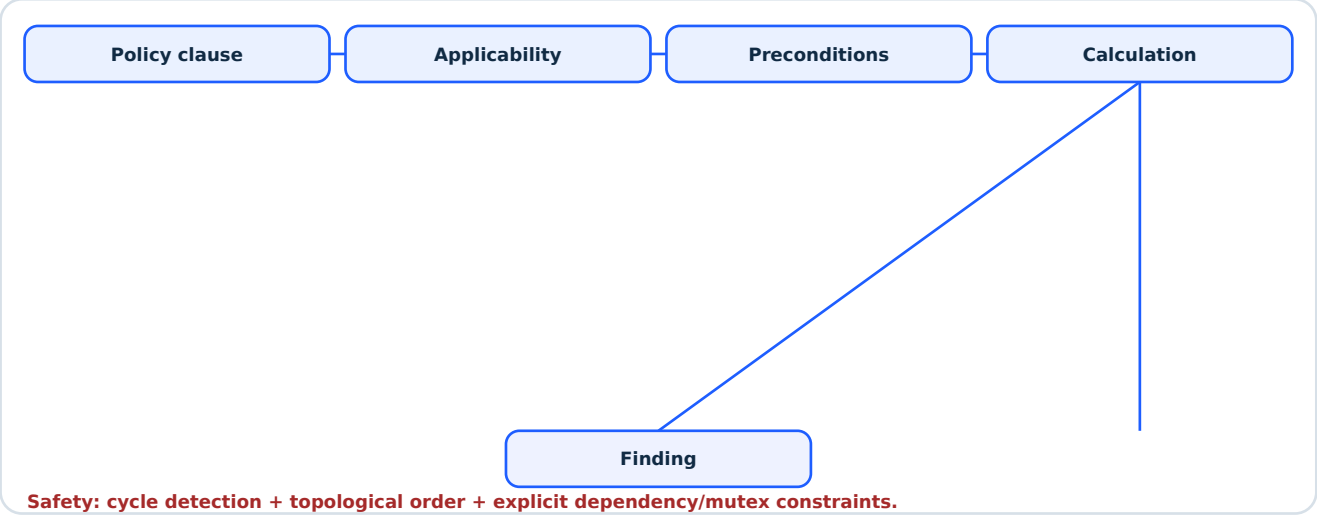
The architecture must not hard-code a universal threshold such as a fixed room-rent percentage when the actual policy/regulation can vary. The applicable threshold must be derived from the evidenced policy context or the configured regulatory rule set.

Clinical boundary

Clinical necessity decisions are outside the deterministic financial scope. When the rejection depends on clinical necessity, the system should identify the boundary and escalate rather than invent a medical judgment.

9. Rule Dependency Graph and FinancialMath

A flat list of rules is unsafe when deductions depend on other deductions, when rules are mutually exclusive, or when the ordering of operations changes the result. ClaimGuard therefore models dependencies explicitly.



Rule Dependency Graph controls

- depends_on: a rule explicitly declares upstream facts or rule results required before execution.
- mutually_exclusive_with: incompatible rules cannot both contribute silently.
- cycle detection: cyclic dependencies fail before adjudication.
- topological ordering: execution follows a deterministic dependency-respecting sequence.
- atomic ledger updates: a deduction changes remaining balance once and is not silently applied again by another finding.

Adjudication state

The internal adjudication model keeps both original amount and remaining balance so that each deduction can be traced to a specific stage. This prevents double deduction and makes reconciliation explicit.

Independent settlement reproduction

ClaimGuard should independently reproduce the insurer settlement from the verified evidence. It should not derive a "recoverable amount" by simply summing every finding's monetary impact. A monetary finding must correspond to an actual discrepancy after applicability, ordering and reconciliation have been evaluated.

10. Adjudication State and Outcomes

Processing lifecycle and authoritative analysis outcome are intentionally separate. A job can finish technically while the claim outcome is still extraction-failed, blocked or review-required.

Queued / running	Pending	No financial result
Completed after extraction failure	EXTRACTION_FAILED	No payout/disputed amount
Completed with missing evidence	INSUFFICIENT_EVIDENCE / NEEDS_REVIEW	Withhold aggregate and per-rule money
Completed with conflict	CONFLICT	Withhold financial decision
Completed and verified	Verified outcome	Show only evidence-backed values

Fail-closed result semantics

- If a required critical document fails extraction, the rule engine is not called for financial adjudication.
- No payout or disputed amount is produced from an incomplete extraction.
- The review explanation identifies the reason for the stop so the next human action is clear.
- The UI never treats processing completion as proof that an adjudication was successfully completed.

11. Frontend and API Safety

The live UI must be a faithful projection of backend-authoritative analysis, not an independent source of synthetic business data.

Current repairs

- Missing API response fields remain null/missing through the normalization layer.
- Mock patient data, estimated billed amounts, synthetic verdicts and fabricated appeal content are removed from the live analysis flow.
- The synthetic one-click Apollo claim loader is removed from the live path.
- Synthetic "completed" extraction-progress messages no longer imply successful model completion.
- High and medium forensic signals route to human review rather than using accusatory fraud language.
- Result money is suppressed whenever a rule verdict requires review or is blocked.
- Per-verdict financial figures are also suppressed when review is pending.
- Portfolio recovery totals remain null or exclude unverified analyses.
- CSV exports do not invent estimated billed amounts.

API/UI contract principle

The backend owns the truth of the analysis outcome. The frontend formats that truth and presents explicit unavailable/review states. It must never infer a missing amount, verdict, confidence score or appeal draft.

12. Security and Document Integrity

ClaimGuard processes untrusted documents containing natural language. A document can contain text that looks like an instruction to the model, a forged page, contradictory information or a credential-like artifact. Those inputs must be treated as data, not as privileged system instructions.

Security controls in the current architecture

- Prompt-injection isolation: extracted document text is untrusted content and must not override system instructions or safety policy.
- Credential hygiene: tracked example files use placeholders rather than credential-shaped values. Any real key previously exposed should be rotated.
- Hash binding: evidence can be tied to exact uploaded document IDs/hashes.
- Forensic semantics: checks that did not execute are marked NOT_ANALYZED rather than scoring as zero.
- Conflict handling: contradictory documents block or escalate rather than being silently merged.

Document integrity and provenance

The architecture is compatible with future stronger document-tampering checks, but the current project should not claim complete PDF-forgery detection merely because provenance and hashing exist. Tampering detection and provenance are complementary controls.

Operational security rule

No production credential should be committed into source control, example configuration, generated logs or evidence fixtures. Example credentials must always be explicit placeholders.

13. Test and UAT Strategy

The architecture is validated as a safety-critical data flow. Regression testing is necessary not only for the final decision but also for proving that dangerous fallback behavior cannot reappear.

Current automated verification

Backend pytest suite	315 passed, 2 skipped
Frontend Node runner	77 passed, 0 failed (35 + 28 + 9 + 5 across tiers)
FastAPI import	Successful
Frontend production build	Successful; large chunk warning from Vite
NPM launcher	Host configuration issue: npm-cli.js missing; tests were run directly with Node

Required adversarial coverage

- Missing credentials, placeholder credentials and provider exceptions.
- Single-page vs multi-page documents, with important values on later pages.
- Critical financial fields missing page, snippet, confidence or document binding.
- Identity-role mismatches, conflicting claim IDs and contradictory dates.
- Leap-year moratorium boundaries and exact threshold mutations.
- Double deduction attempts and rule-order changes.
- Cyclic dependencies and mutually exclusive rules.
- Prompt injection in rejection letters or uploaded documents.
- Clinical-necessity rejection paths.
- OCR degradation, unusual PDF layouts and low-confidence extraction.
- Frontend/API missing-field normalization and removal of demo fallbacks.

Golden/UAT principle

A Golden UAT should derive expected values independently from the application logic. Expected discrepancy amounts must not be hard-coded into the test simply to make the application appear correct.

14. Current Implementation Status

The 01 Oct 2026 safe-analysis audit closed the identified fail-open paths and preserved the existing working architecture rather than rewriting unrelated components.

Extraction fallback	Implemented	Failure returns no structured data and explicit extraction-failure/review state.
Multi-page PDF extraction	Implemented	All rendered pages are handed to multimodal extraction.
Adjudication fail-closed	Implemented	Rule engine bypassed when required extraction is unsuccessful.
Critical financial provenance	Implemented	Page + matching snippet + confidence + document binding required.
UI fallback removal	Implemented	Missing API data stays missing; mock financials and demo loaders removed.
Review-state money suppression	Implemented	Aggregate and per-rule financials withheld when review/blocked.
Audit-chain empty state	Implemented	Empty chain is treated as unverified.
Forensic not-run state	Implemented	Displays NOT_ANALYZED rather than zero score.
Security placeholder hygiene	Implemented	Credential-shaped value replaced in .env.example; rotate any real key if previously exposed.
Live provider validation	Pending	Real VLM extraction with real credentials/documents is not yet verified.
Broader provenance	Pending	Policy-clause and nonfinancial provenance still require validation.

Repository state

No commit was made during the safe-analysis repair. Existing local changes were preserved. This is an important operational safeguard when the working tree contains unrelated or pre-existing engineering work.

15. Research Grounding

The current architecture is informed by a layered set of research references. The core methodological pattern is to combine AI-based document understanding with deterministic verification instead of treating a language model as the final adjudicator.

Owolabi (2025)	Denial/appeal analysis as the closest problem-domain foundation; ClaimGuard extends from appeal triage to evidence-based rejection verification.
Peng et al. (2021)	Insurance information extraction and normalization from unstructured interactions.
Çavuşoğlu et al. (2018)	Insurance policy/table key extraction and preservation of structured policy information.
PICK (2020)	Complex document KIE using text, visual and layout relationships.
LayoutLMv3 (2022)	Multimodal document understanding for text/image/layout-rich insurance documents.
Berger et al. (2023)	Automated regulatory compliance verification as a methodological analogue.
Compliance-to-Code	Transform compliance clauses into executable deterministic logic - closely aligned with ClaimGuard rule engineering.
Gupta et al. (2021)	Insurance fraud/anomaly analysis as supporting context, not the primary ClaimGuard objective.
Wang et al. (2025)	Interpretable insurance analytics and the need for explainable model outputs.
AGB-DE (2024)	Demonstrates the difficulty of automated legal-clause assessment and motivates uncertainty handling.
LegalPro-BERT (2024)	Domain-specific legal provision classification.
Goda (2019)	Indian insurance Ombudsman/policyholder protection context.
Redressal Mechanism study (2025)	Indian grievance and claim-rejection context motivating better evidence preparation.
Grobler et al. (2024)	Document integrity and PDF tampering context supporting provenance/hashing controls.
Mansoori et al. (2023)	Healthcare ML context; supporting rather than central to ClaimGuard.

16. Next Engineering Phase

The current repairs establish a safer architecture, but the next phase should focus on proving that the safety properties hold under realistic provider behavior and diverse documents, not just in mocks or unit tests.

Priority 1 - Live VLM/provider validation

- Run real multi-page policies, hospital bills and rejection/settlement letters through the configured provider.
- Verify extraction failure behavior when credentials are missing, expired or invalid.
- Measure extraction confidence and compare cited snippets to actual OCR/source text.
- Verify that later policy pages containing exclusions, waiting periods or limits are captured.

Priority 2 - Expand provenance coverage

- Extend source binding from financial values to policy clauses, dates, identifiers and key nonfinancial evidence.
- Make clause citations and policy-section references first-class evidence objects.
- Validate that every UI finding can trace back to the source facts used to generate it.

Priority 3 - Realistic UAT corpus

- Build a frozen synthetic benchmark of approximately 50-100 cases spanning correct rejections, incorrect rejections, borderline cases, identity conflicts, low-confidence extraction, conflicting documents, portability/moratorium, duplicate deductions, clinical rejections, contradictory clauses, OCR degradation, prompt injection and policy-version changes.
- Derive expected outputs independently and record the acceptable outcome state: verified, review, conflict or extraction failure.
- Run the full browser/VLM workflow, not only unit tests.

Priority 4 - Performance and production hardening

- Address the frontend large-chunk warning where practical.
- Repair the host npm launcher configuration so standard project commands work without workarounds.
- Add observability for extraction latency, provider failure rates, evidence completeness and rule-review rates.
- Keep production financial transactions outside the current autonomous scope until end-to-end validation and governance are established.

Release posture after this plan

Current posture: safe-analysis architecture implemented and regression-tested, with explicit remaining limitations. Next release gate: real-document/provider validation plus expanded provenance and frozen end-to-end UAT. The project should continue to use the language "validated with limitations" until those gates are completed.

17. Architecture Summary

ClaimGuard is designed as a controlled evidence-verification pipeline rather than a free-form AI adjudicator. The system starts with multiple claim documents, uses AI to read and structure them, applies Tier 0 safety gates, records provenance, determines applicable policy conditions, executes a dependency-aware deterministic rule graph, reproduces financial calculations independently, and presents either an evidence-backed result or an explicit review/block state.

The repaired architecture specifically closes the most dangerous failure mode identified in the latest audit: incomplete extraction can no longer silently enter financial rules and appear as a successful adjudication. The same principle extends to the frontend: missing backend data remains missing instead of becoming demo data, estimated amounts, invented verdicts or synthetic appeal text.

Final engineering principle

Evidence first. Applicability before rules. Deterministic math for money. Provenance for every important fact. Human review for uncertainty. No fabricated completion.

End of document